CHAPTER 14

Database Programming

Chapter Overview

14.1 Database Management System

A database management system is a collection of programs that are used to create and maintain databases. Database is a collection of data in an organized way. Database consists of tables, which further consist of rows and columns.

Tables

Table is the fundamental object of the database structure. The basic purpose of a table is to store information. Each table consists of rows and columns. A table is a very convenient way to store information. You can easily manipulate the data in a table. All the information in an RDBMS is stored in tables.

Serial No	Name	Qualification	Email
1	Usman	B.Sc.	usman@itseries.com.pk
2	Tariq	M.Sc.	tariqmj@hotmail.com
3	Imran	M.Sc.	imran735@hotmail.com

Table: Records table

Rows

Rows are the horizontal part of the table. It is a collection of related fields. For example in the above table, we have three rows. Each row contains a record of different person.

2	Usman	B.Sc.	usman@itseries.com.pk

A single row / record

Columns

Columns are the vertical part of the table. For example, in the above table, all values under "Name" heading make a column.

Name
Usman
Tariq
Imran

A single column

14.2 Relational Databases

Relational database is a type of database in which different tables are connected with one another. This connection is known as relationship. Table in relational database is also called **relation**. This relationship is very useful for creating an efficient database.

14.2.1 Keys in Relational Databases

Databases are used to store data in tables. This data is manipulated for different purposes as and when required. For example, you may need to search or delete a particular record from the table etc. You have to identify each data item from others. It means that the data should be stored in databases in such way that it may be used easily later. Keys are used to identify different records in databases.

- Keys ensure that each record in the table is properly identified.
- They help in establishing and enforcing reliability.
- They help in establishing relationships between tables.
- They enable faster searches in a table.

1. Primary Key

A primary key is the column or a group of columns that can uniquely identify a row in a table. Some important points of a primary key are as follows:

- It must uniquely identify each record in a table.
- It must contain unique values.
- It cannot have a null value.
- Each table can have only one primary key.

2. Foreign Key

A foreign key is a copy of a primary key in another table. The key connects to another table when a relationship is being established between two tables. A table may contain many foreign keys. Some important points of a primary key are as follows:

- It has the same name as the primary key from which it was referred.
- The field specifications of the primary key from which it was copied have to be the same for the foreign key too.

14.3 Relationships

Different tables in a database can be joined together by establishing relationship between them. Different types of relationships are as follows:

- One-to-One
- One-to-Many
- Many-to-Many

14.3.1 One-to-One Relationship

This type of relationship is used when "for each record in first table, there is only one record in the second table and for each record in second table, there is only one record in the first table". Suppose there are two tables named **Student** and **Marks**. Student table contains basic information of students and Marks table contains the marks of the students. Both tables are joined with one-to-one relationship. It means that for each record in Student table, there is only one record in Marks table and vice versa. Each Roll No occurs once in both tables.

Roll No	Name	Address
1	Usman	Faisalabad
2	Adnan	Faisalabad
3	Faisal	Lahore
4	Nadeem	Islamabad

Student table

Roll No	Marks	Grade
1	100	A
2	50	C
3	68	B
4	84	A

Marks table

14.3.2 One-to-Many Relationship

This type of relationship is used when "for each record in first table, there are one or more records in the second table and for each record in second table, there is only one record in the first table".

For example, we have two tables named Student and Subject. Student table contains the basic information of the students and Subject table contains the marks of the students in different subjects. Both tables are joined with one-to-many relationship. It means that for each record in Student table, there can be one or more record in Subject table and for each record in Subject table, there can be only one record in Student table. There are more records in Subject table against one record in Student table because one student studies many subjects.

Roll No	Name	Address
1	Usman	Faisalabad
2	Adnan	Faisalabad
3	Faisal	Lahore
4	Nadeem	Islamabad

Student table

Roll No	Subject	Marks
1	English	58
1	Math	88
1	Computer	92
2	English	49
2	Math	72
2	Computer	69
3	English	51
3	Math	37
3	Computer	63
4	English	59
4	Math	64
4	Computer	77

Subject table

14.3.3 Many-to-Many Relationship

This type of relationship is used when "for each record in first table, there are one or more records in the second table and for each record in second table, there are one or more records in the first table".

For example, we have two tables named Authors and Books. Authors table contains the information of the authors and Books table contains the information of the books. Both tables are joined with many-to-many relationship. It means that for each record in Author table, there can be one or more records in Books table and vice versa. It means that one author may have written one or more books. Similarly, one book may have been written by one or more authors.

14.4 Database Applications in Visual Basic

Visual Basic provides many facilities for creating different types of databases applications. You can create front-end part of the applications in Visual Basic. A standard database application consists of user interface, database engine and data store.

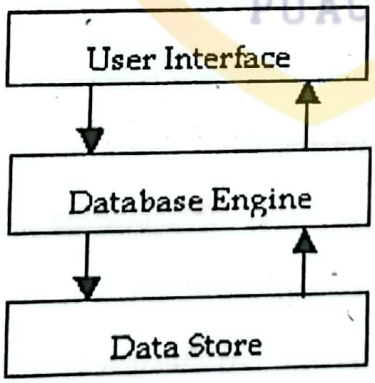


Figure: Three parts of database applications

User-Interface

The user interface is the part of application that is used by the user. It is also called the front-end. The user interacts with the application through graphical user interface. It contains different forms to display data. The user can view or update data by using these forms. These forms contain Visual Basic code that is executed on the request of the user. For example, the user may add, delete or search different records from database.

The requests of the user are not sent directly to the physical database but to the database engine. The database engine performs the requested operations on the data store and returns the desired results to the application.

2. Database Engine

Database Engine receives the requests of the user from user-interface and translates them into physical operations on the data store. The database engine handles all operations on the database. The database engine lies between our application and the physical database.

3. Data Store

The data store is the database tables themselves. It can be Microsoft Access database or any other databases. The application might access data stored in several different database files and formats. Data Store contains all the data to be used by your application.

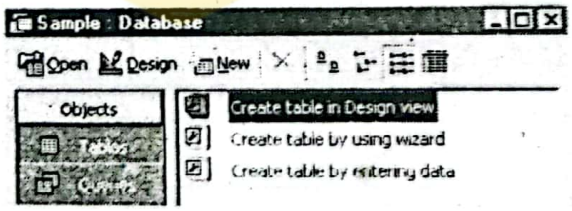
14.5 Creating Database in MS Access

Microsoft Access is an RDBMS used to create client/server applications. It is used popularly with Visual Basic. You can create a database in MS Access as follows:

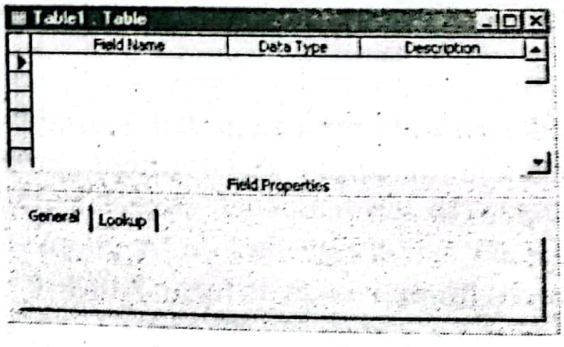
Practical 14.1

1. Start Microsoft Access. A dialog box will appear containing different options.
2. Select the first option **"Blank Access database"** and click **OK**.

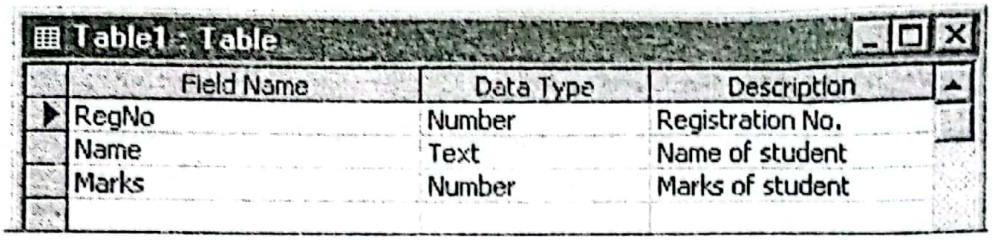
3. **File New Database** dialog box will appear. Give any name for database and click **Create**. A new database will be created and the following window will appear.



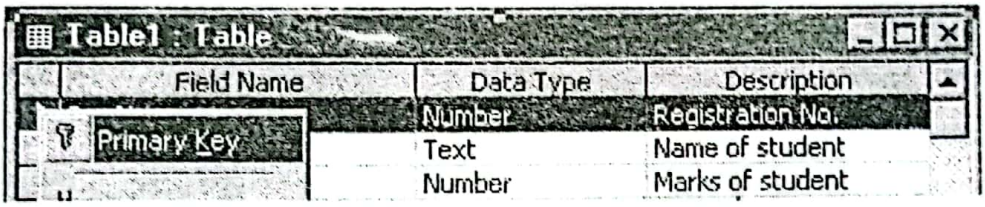
4. Double click on **Create table in Design view**. The following window will appear



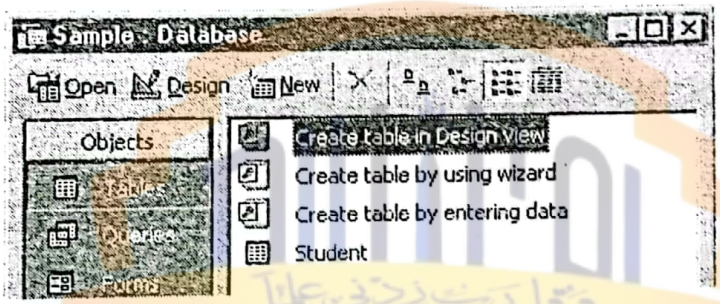
5. Enter the following fields in the window.



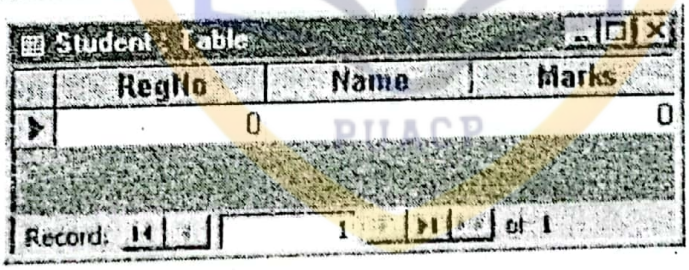
6. Right click on **RegNo** field and click on **Primary Key** from the popup menu. A sign of key will appear on the field and it will become primary key of table.



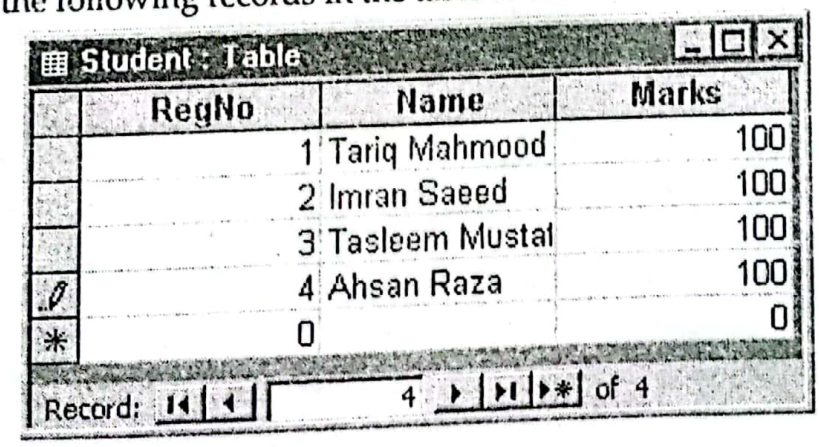
7. Select **Save** from **File** menu or press **CTRL+S**. A box will appear.
8. Enter any name of table and press **OK**. The table will be saved.
9. Close the window. The name of the table will appear in the main window.



10. Double click the Student table. The following window will appear where you can enter any number of records in the table.



11. Enter the following records in the table and close the window.



12. Exit Microsoft Access.

You have created a database "Sample" with one table "Student". You can create as many tables as required in the database by following the same procedure as above.

14.6 Designing the User Interface

The user interface is an important part of a database application. The user interacts with database through this interface. You can create user interface by using different control available in the toolbox of Visual Basic.

14.6.1 Data Control

Data control is a powerful tool in Visual Basic that is used to connect the user interface with the database. You can access and manipulate database using this control. The data control is attached to a specific table in a database. The data control itself does not display data. It works as a link between the front-end and the database.

Figure: Navigation through Data control

14.6.2 Properties of Data Control

Some important properties of Data control are as follows:

Connect

This property is used to specify the type of database. You can connect to different types of database like Access, dBase and FoxPro etc.

DataBaseName

This property is used to specify the name and location of the database.

RecordSource

This property is used to specify the name of the table in the database

RecordSetType

This property is used to specify the type of the recordset. Different types of recordset are as follows:

RecordSet Type	Description
Dynaset	A dynamic set of records can add, change, or delete records from a dynaset-type Recordset, and the changes will be reflected in the underlying tables.
Table	A set of records that represents a single database table that can be used to add, change, or delete records.
Snapshot	A copy of a set of records that can be used to find data or generate reports but cannot be updated.

BOFAction

You can press **Move Previous** button or **Move First** button to reach the first record. When you reach the first record, **BOF** property becomes **True**. If you again press **Move Previous** button, an error will occur. **BOFAction** property is used to specify the action to be executed this time to avoid the error.

Different values for this property are as follows:

Value	Description
0 – Move First	The control pointer remains on first record.
1 – BOF	The control pointer remains on first record and Move Previous button is disabled.

EOFAction

You can press **Move Next** button or **Move Last** button to reach the last record. When you reach the last record, **EOF** property becomes **True**. If you again press **Move Next** button, an error will occur. **EOFAction** property is used to specify the action to be executed at this time to avoid the error. Different values for this property are as follows:

Value	Description
0 – Move Last	The control pointer remains on last record.
1 – EOF	The control pointer remains on last record and Move Next button is disabled.
2 – AddNew	The table becomes ready to add new record.

ReadOnly

This property is used to specify whether the table can be changed or not. It is a Boolean property.

Value	Description
False	You can make changes to database.
True	You cannot make any change to database.

14.6.3 Methods of Data Control

Some important methods of Data control are as follows:

Refresh

This method is used to refresh the contents of data control. If reloads the data control with the latest records from the table.

UpdateRecord

This method is used to update the modified records. If the user has made any changes to the record, the change is updated in the table.

14.7 Data Bound Controls

All controls that can work with the data control to access data from a database are called data-bound controls. The process of attaching a data-bound control to a data control is called **binding**.

The bound control is associated with a particular field of a table in database. For example, a text box that displays the name of a student is bound to the **Name** field in the **Student** table.

14.7.1 Properties of Data Bound Controls

Some important properties of Data bound controls that are related to databases are as follows:

DataSource

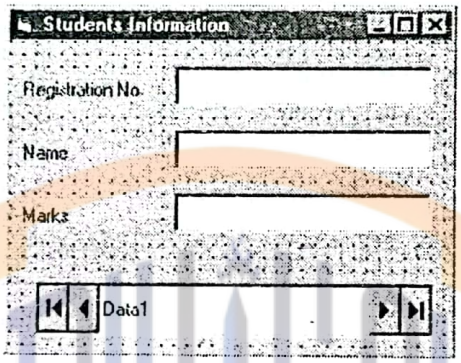
This property is used to specify the name of the data control that is connected to a table of a database.

DataField

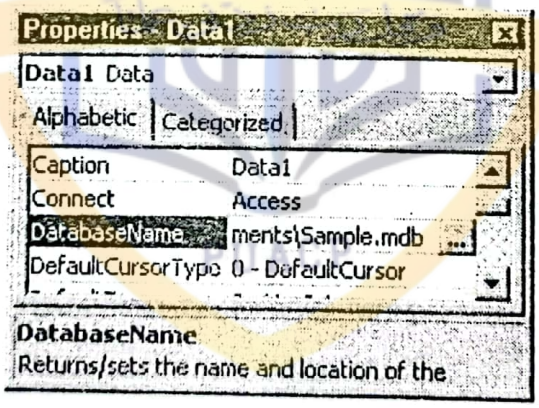
This property is used to specify the name of the field of a table for which the data bound control is used.

Practical 14.2

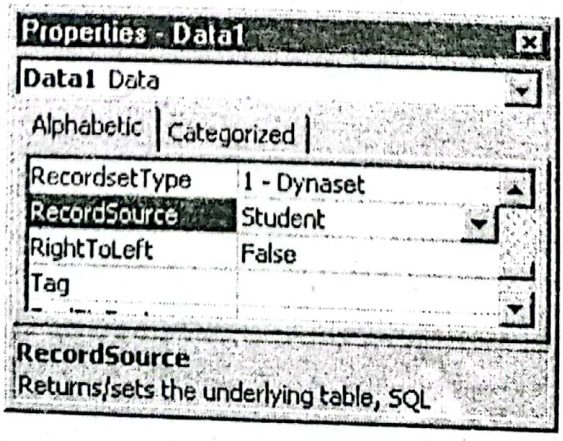
1. Start a new Standard EXE project.
2. Place three labels, three textboxes and one Data control on the form.



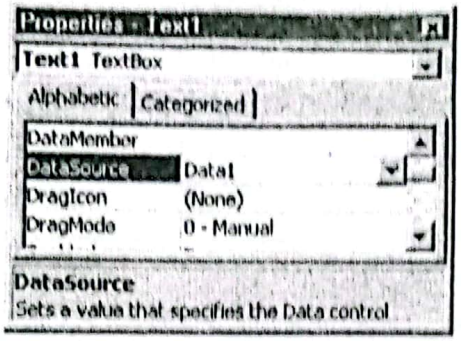
3. Click on Data control and select Sample database in DatabaseName property.



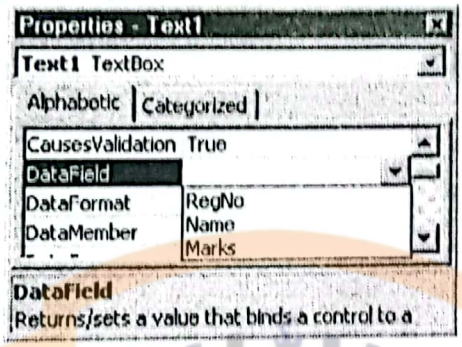
4. Select **Student** table from **RecordSource** property.



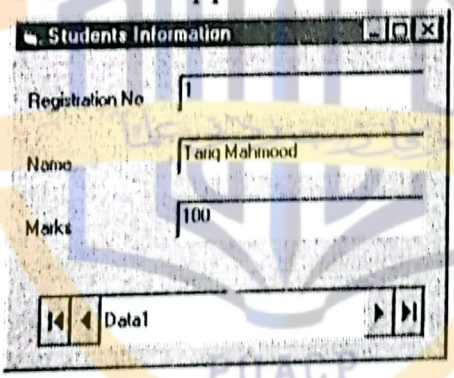
5. Select all three textboxes and select **Data1** from **DataSource** property.



6. Set the **DataField** property of **Text1** to **RegNo.**
7. Set the **DataField** property of **Text2** to **Name.**
8. Set the **DataField** property of **Text3** to **Marks.**



9. Run the project. The form will appear as follows:



You can navigate through all records of table by clicking the buttons of data control.

Note: The Data control does not work with the files of Microsoft 2000 or later. The Student database must be converted to prior version by using the following procedure:

- Select Tools > Database Utilities > Convert Database > To Access 97 File Format... A dialog box will appear to get new file name.
- Select a new location for **Student** database and click **Save** button. The database will be converted to Access 97 format and saved at the new location.

14.8 RecordSet Object

The **RecordSet** object is used in Visual Basic to access the data in database. The data retrieved from the table is stored in **Recordset** object. When you set the **RecordSource** property of Data control, a **Recordset** object is created automatically.

Visual Basic uses this object to manipulate the data. We can also access this object in code to utilize its properties and methods for accessing and manipulating data. Different types of **Recordset** object are as follows:

1. Table Type Recordset

This type is used to reference a specific database table. It allows the user to add, delete or modify records in the table. It uses **Seek** method to search a record in the table. **Seek** method works only if you have created indexes in your database.

2. Dynaset Type Recordset

This type is a dynamic set of records that may consist of different field of one or more tables. It can be a single table, part of a table or a collection of data from multiple tables. It uses **Find** method to search a record in the table.

3. Snapshot Type Recordset

This type is a static copy of a set of records from one or more tables. It cannot be updated. It also uses **Find** method to search a record in the table.

14.8.1 Properties of Recordset Object

Some important properties of Recordset object are as follows:

RecordCount

This property is used to determine the total number of records in a table.

NoMatch

This is a Boolean property and is used to determine whether the search was successful or not.

Value	Description
True	The search was not successful.
False	The search was successful.

BOF

This is a Boolean property. It returns **true** if the record pointer reaches beginning of recordset. Otherwise it returns **false**.

EOF

This is a Boolean property. It returns **true** if the record pointer reaches end of recordset. Otherwise it returns **false**.

14.8.2 Methods of Recordset Object

Some important methods of Recordset object are as follows:

MoveFirst

This method is used to move the record pointer to the first record in the recordset.

```
Data1.Recordset.MoveFirst
```

MoveLast

This method is used to move the record pointer to the last record in the recordset.

```
Data1.Recordset.MoveLast
```

MoveNext

This method is used to move the record point to the next record in the recordset.

```
Data1.Recordset.MoveNext
```

MovePrevious

This method is used to move the record pointer to the previous record in the recordset.

```
Data1.Recordset.MovePrevious
```

AddNew

This method is used to add new record to the database. When this method is executed, all data-bound controls on the form become empty. After the user entered new values in the data-bound controls, we need to execute **Update** method to store these values permanently in the database.

```
Data1.Recordset.AddNew
```

Update

This method is used to update the values permanently in the database. It is used with **AddNew** method.

```
Data1.Recordset.Update
```

Edit

This method is used to edit the current record in database. After changing the current record, **Update** method should be executed to make these changes permanent in database.

```
Data1.Recordset.Edit
```

Delete

This method is used to delete the current record from the database. When you delete a record from the database, the values in data-bound controls are not erased automatically. You should move the record pointer to some other record after deleting the current record.

```
Data1.Recordset.Delete
```

FindFirst

This method is used to search a record that matches with the given criteria. If more than one records match with the criteria, it returns the first of them.

```
Data1.Recordset.FindFirst <Criteria>
```

FindLast

This method is used to search a record that matches with the given criteria. If more than one records match with the criteria, it returns the last of them.

```
Data1.Recordset.FindLast <Criteria>
```

FindNext

This method is used to search a record that matches with the given criteria. It looks in only records that come after the current record in the recordset. Suppose we have ten records and the record pointer is on 4th record. The first three records will not be included in search.

```
Data1.Recordset.FindNext <Criteria>
```

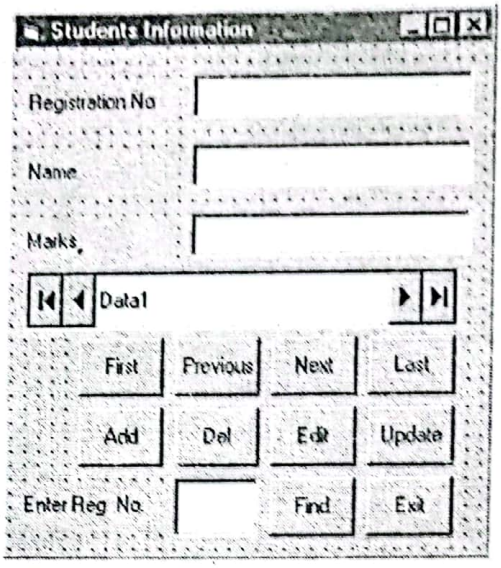
FindPrevious

This method is used to search a record that matches with the given criteria. It looks in only records that come before the current record in the recordset. Suppose we have ten records and record pointer is on 4th record. The last six records will not be included in search.

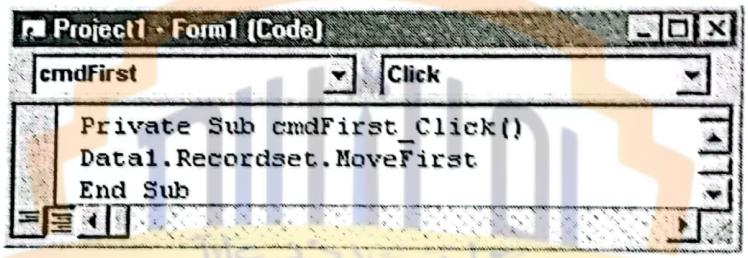
```
Data1.Recordset.FindPrevious <Criteria>
```

Practical 14.3

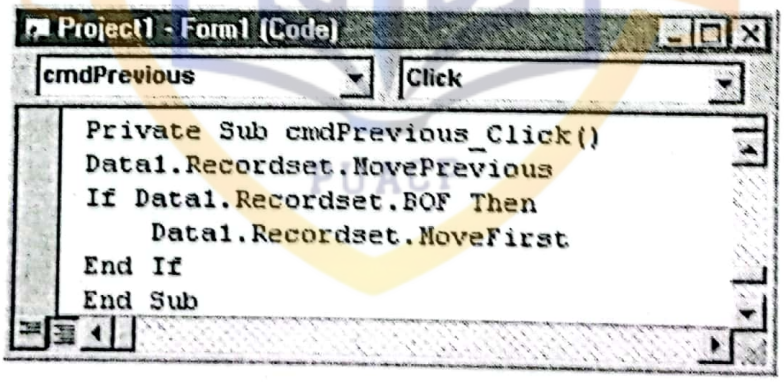
1. Use the form of previous example and place eight buttons and change their Captions as follows:



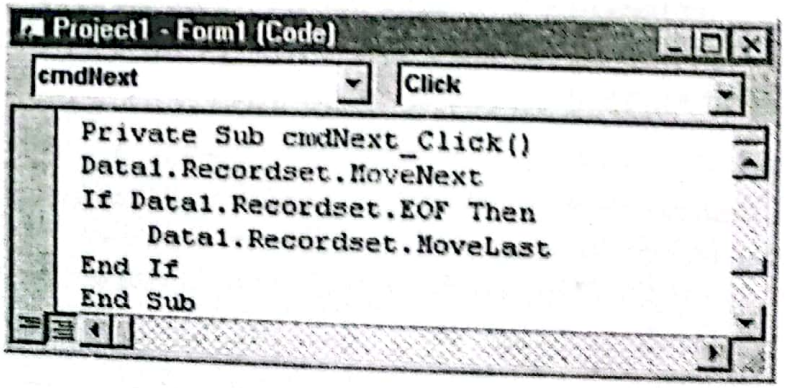
2. Add the following code in **Click** event of the **First** button:



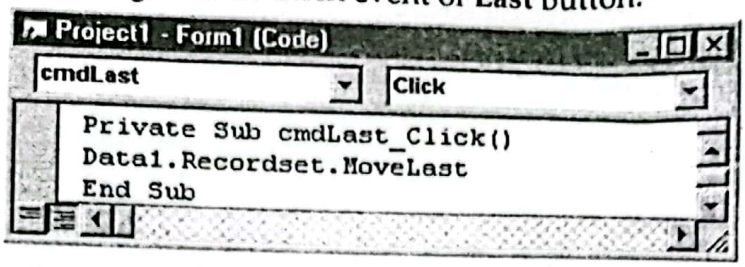
3. Add the following code in **Click** event of the **Previous** button:



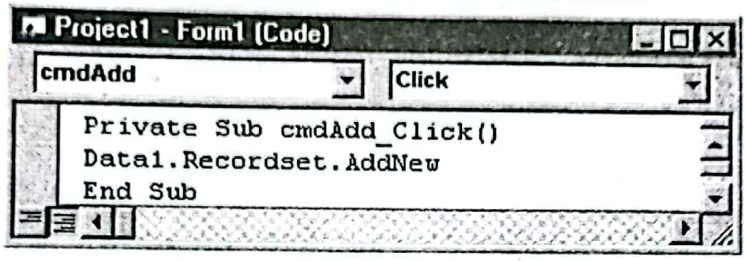
4. Add the following code in **Click** event of the **Next** button:



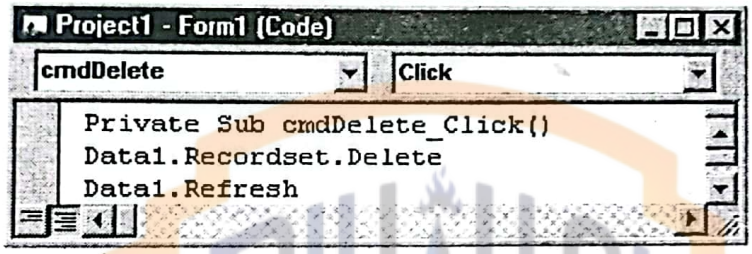
5. Add the following code in **Click** event of **Last** button.



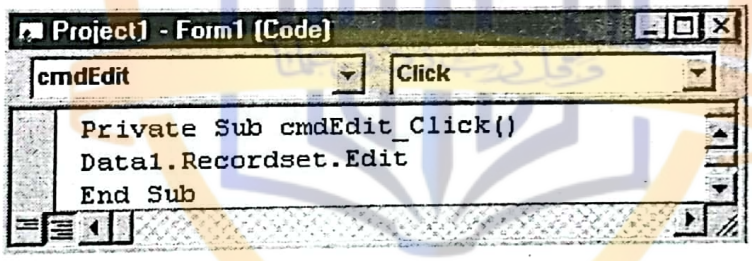
6. Add the following code in **Click** event of **Add** button.



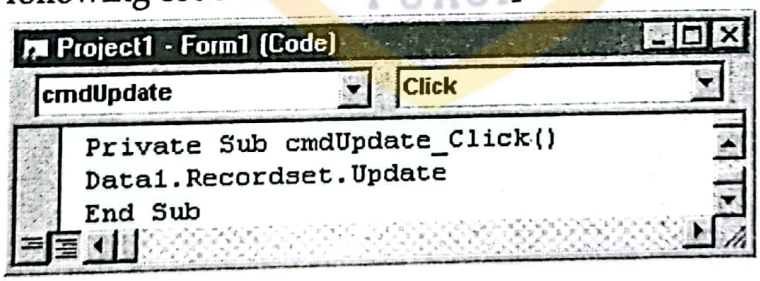
7. Add the following code in **Click** event of **Delete** button.



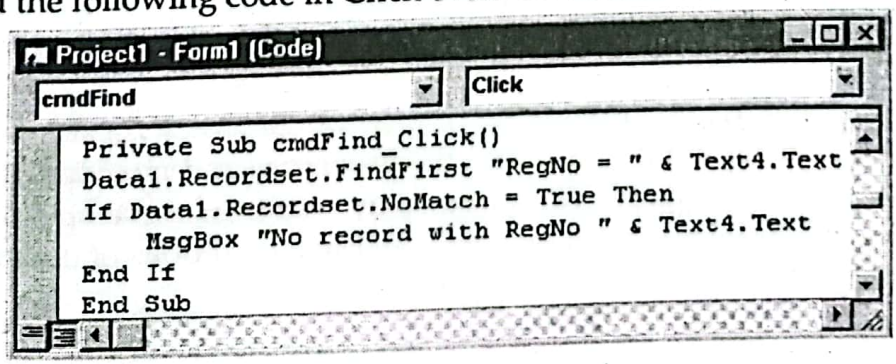
8. Add the following code in **Click** event of **Edit** button.



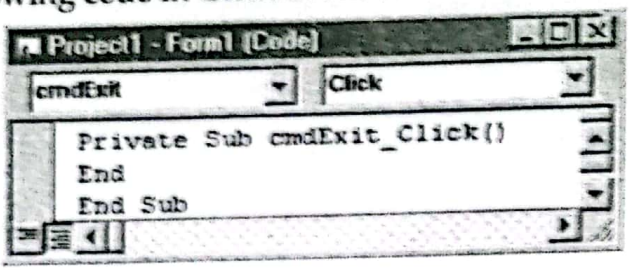
9. Add the following code in **Click** event of **Update** button.



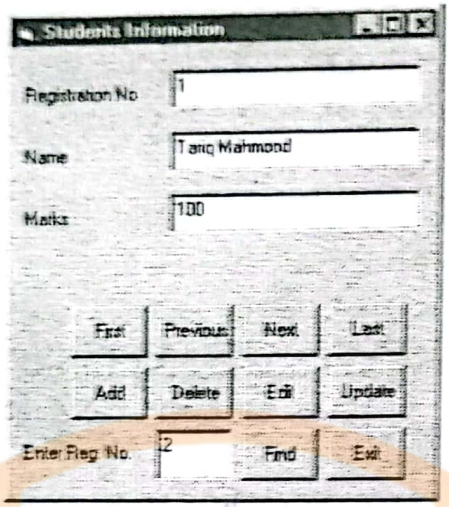
10. Add the following code in **Click** event of **Find** button.



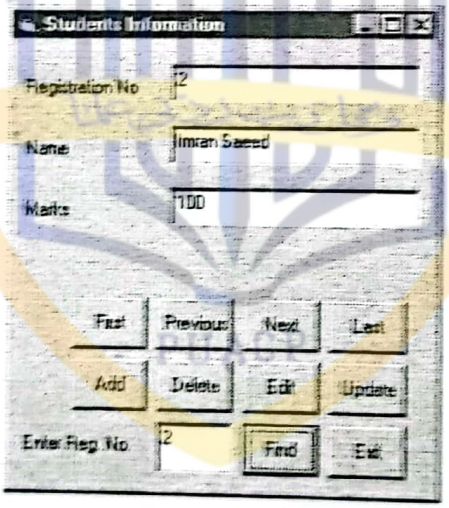
11. Add the following code in Click event of Exit button.



12. Run the project and Enter 2 in the Text4.



13. Click **Find** button. The records of RegNo 2 will appear in the form.



14.9 Data Access Object

The DAO model is a collection of object classes that model the structure of a relational database system. They provide properties and methods for creating databases, defining tables, fields and indexes, establishing relations between tables, navigating and querying the database, and so on.

Database programming in Visual Basic consists of creating data access objects, such as Database, TableDef, Field, and Index. You use the properties and methods of these objects to perform operations on the database. You can display results of these operations and accept input from the user on Visual Basic forms. There are three categories of databases that Visual Basic recognizes through DAO and the Jet engine:

14.9.1 Visual Basic Databases

Visual Basic databases are also called **native databases**. These database files use the same format as Microsoft Access. These databases are created and manipulated directly by the Jet engine and provide maximum flexibility and speed.

14.9.2 External Databases

These are **Indexed Sequential Access Method (ISAM)** databases in several popular formats, including Btrieve, dBASE III, dBASE IV, Microsoft FoxPro versions 2.0 and 2.5, and Paradox versions 3.x and 4.0. You can create or manipulate all of these database formats in Visual Basic. You can also access text file databases and Microsoft Excel or Lotus 1-2-3 worksheets.

14.9.3 ODBC Databases

These include client-server databases that conform to the ODBC standard, such as Microsoft SQL Server. To create true client-server applications in Visual Basic, you can use ODBCDirect to pass commands directly to the external server for processing.

14.10 DAO Hierarchy

DAO objects are organized in a hierarchical relationship. Objects contain collections, and collections contain other objects. The **DBEngine** object is the top-level object that contains all the other objects and collections in the DAO object hierarchy. The following table summarizes the DAO objects.

Object	Description
Database	Open database
DBEngine	The top-level object in the DAO object hierarchy
Error	Data access error information
Field	Field in a TableDef, QueryDef, Recordset, Index or Relation object
Group	Group account in the current workgroup
Index	Table index
Parameter	Query parameter
Property	Property of an object
QueryDef	Saved query definition in a database
Recordset	Set of records defined by a table or query
Relation	Relationship between two table or query fields
TableDef	Saved table definition in a database
User	User account in the current workgroup
Workspace	Active DAO session

Table: DAO objects

Setting the Reference to DAO Object Library

To work with DAO objects from within any application, you must have a reference to the Microsoft DAO 3.5 object library. To set a reference to the Microsoft DAO object library, open Visual Basic, click **References** on the **Tools** menu, and then select the **Microsoft DAO 3.6 Object Library** check box.

14.10.1 The DBEngine Object

The DBEngine object is the top-level object in the DAO object hierarchy. It contains all other DAO objects and collections. The DBEngine object is the default object in the object model, so you do not need to refer to it explicitly.

The DBEngine object contains two collections:

- Workspaces collection
- Errors collection.

The Workspaces collection is the default collection of the DBEngine object, so you do not need to refer to it explicitly. You can return a reference to the first Workspace object in the Workspaces collection of the DBEngine object in any of the following ways:

```
Set wrk = DBEngine.Workspaces(0)
Set wrk = DBEngine(0)
Set wrk = Workspaces(0)
```

14.10.2 Workspace Object

A workspace, represented by a **Workspace** object, is an active session for a particular user account. A session represents a sequence of operations performed by the database engine. A session begins when a particular user logs on and ends when that user logs off. The operations that a user can perform during a session are determined by the permissions granted to that user. If you do not specifically create a workspace, then DAO creates a default workspace for you.

You can use the default workspace of Jet engine, which is created automatically when Jet engine is accessed. The following example declares an object variable "MyWS" and sets its reference to the default workspace of Jet Engine.

```
Dim MyWS As Workspace
Set MyWS = DBEngine.Workspaces(0)
```

Creating a New Microsoft Jet Workspace

You can create a new Microsoft Jet workspace by using the **CreateWorkspace** method of the DBEngine object as follows.

```
Dim wrk As Workspace
Set wrk = CreateWorkspace("JetWorkspace", "Admin", "", dbUseJet)
```

14.10.3 Database Object

The **Database** object represents an open database. It can be a **Microsoft Jet** database or an external data source. The **Databases** collection contains all currently open databases.

Opening a Database Object

You can a database and return a reference to the **Database** object by using the **OpenDatabase** method of the **DBEngine** object or of a **Workspace** object. When you use the **OpenDatabase** method of the **DBEngine** object, **Microsoft DAO** opens the database in the default workspace, as shown in the following example.

```
Dim dbs As Database
Set dbs = OpenDatabase("DatabaseName")
```

Creating a Database Object

You can use the **CreateDatabase** method of Workspace object to create a new database.

```
Dim MyDB As Database
Set MyDB = MyWS.CreateDatabase("C:\DAO.mdb", dbLangGeneral, dbVersion30)
```

The constant dbVersion30 specifies a Jet version 3.0 database. If you use the dbVersion30 constant to create a version 3.0 database, only 32-bit applications using the Jet version 3.0 engine or higher will be able to access it.

Closing a Database

You can close a database by calling its **Close** method.

```
MyDB.Close
```

14.10.4 TableDef object

A **TableDef** object represents the stored definition of a table in a **Microsoft Jet** workspace. The **TableDefs** collection contains all stored **TableDef** objects in a database.

Creating a TableDef Object

The **CreateTableDef** method of the **Database** object is used to create new **TableDef** objects for each table in the database.

```
Dim MyTab As TableDef
Set MyTab = MyDB.CreateTableDef("Authors")
```

Adding a Table to Database

The **Append** method of **Database** object is used to add the table to database.

```
MyDB.TableDefs.Append MyTab
```

Deleting a Table

You can delete a table by calling the **Delete** method of the **TableDefs** collection. Deleting a table deletes all the fields, indexes, and data contained within the table. The following example deletes the Authors table:

```
MyDB.TableDefs.Delete "Authors"
```

14.10.5 Field object

Use the **CreateField** method of the **TableDef** object to create new **Field** objects for each field in the table. You can set properties of each field to define the field's size, data type, and other needed attributes. The following example creates two fields.

```
Dim MyFld(1) As Field
Set MyFld(0) = MyTab.CreateField("Au_ID", dbLong)
MyFld(0).Attributes = dbAutoIncrField
Set MyFld(1) = MyTab.CreateField("Author", dbText)
MyFld(1).Size = 50
```

Adding a Filed to Table

You can add a field to an existing table by appending a field to an existing **Fields** collection. The **Append** method is used to add each field to its table.

```
MyTab.Fields.Append MyFld(0)
MyTab.Fields.Append MyFld(1)
```

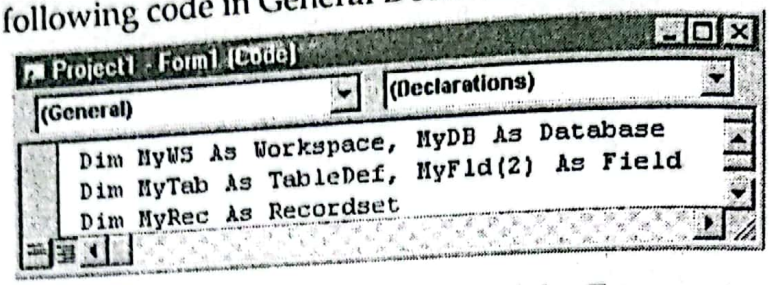
Deleting a Filed

You can delete a **Field** only if it is not a part of any **Index** objects. To delete an individual **Field**, use the **Delete** method of the **TableDef** object.

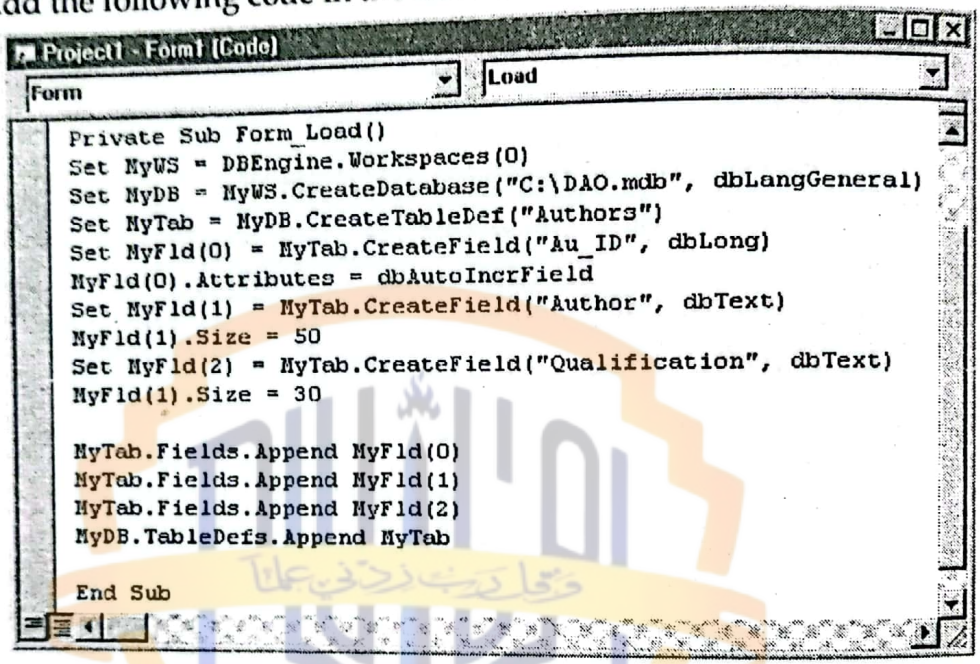
```
MyTab.Fields.Delete "Author"
```

Practical 14.4

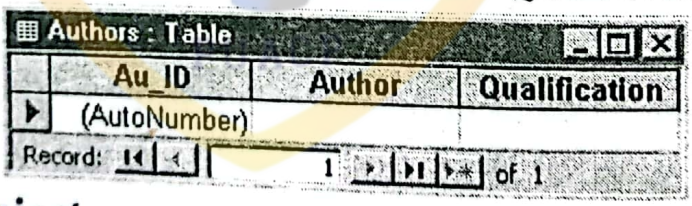
1. Start a new Standard EXE project.
2. Add the following code in General Declaration section.



3. Add the following code in the **Load** event of the Form.



4. Run the project. The code will create a database on C: drive with one table **Authors** with three fields **Au_ID**, **Author** and **Qualification** as follows:



14.10.6 Index Object

You can add indexes to the **TableDef** by creating them with the **CreateIndex** method. In order to specify which fields are to be indexed, you create new **Field** objects with the **CreateField** method of the **Index** object.

```
Dim MyIdx As Index
Set MyIdx = MyTab.CreateIndex("Index1")
MyIdx.Primary = True
MyIdx.Unique = True
```

Now create fields for the index object using **CreateField** method of **Index** object:

```
Set NewFld = AuIdx.CreateField("Au_ID")
```

Adding an Index to Table

Now append the field to the **Index** and the Index to the **TableDef** object by using the **Append** method of the Index object.

```
AuIdx.Fields.Append NewFld
MyTd.Indexes.Append AuIdx
```

The fields created with the **CreateField** method of the **Index** object are not added to the **TableDef**. Instead, they are added to the **Index** object, and given the same **Name** property as the **TableDef** field that they are to index.

Deleting an Index

The syntax for deleting an index is similar to that used to delete a table. The code in the following example removes "Index1" index from the **Indexes** collection in the **Authors** table:

```
MyDB.TableDefs("Authors").Indexes.Delete "Index1"
```

14.10.7 QueryDef Object

The **QueryDef** object represents a query in DAO. The **QueryDefs** collection contains all **QueryDef** objects that are saved with your database and any temporary **QueryDef** objects that are currently open.

Creating a QueryDef Object

You can create a query with DAO by using the **CreateQueryDef** method of a **Database** object, as shown in the following example.

```
Dim MyDB As Database, MyQ As QueryDef
Dim strSQL As String
strSQL = "SELECT * from Authors WHERE Au_ID <= 10;"
Set MyDB = OpenDatabase("IT")
Set MyQ = MyDB.CreateQueryDef("Authros", strSQL)
```

14.10.8 Recordset Object

The **Recordset** object represents a set of records within your database. The **Recordsets** collection contains all open **Recordset** objects.

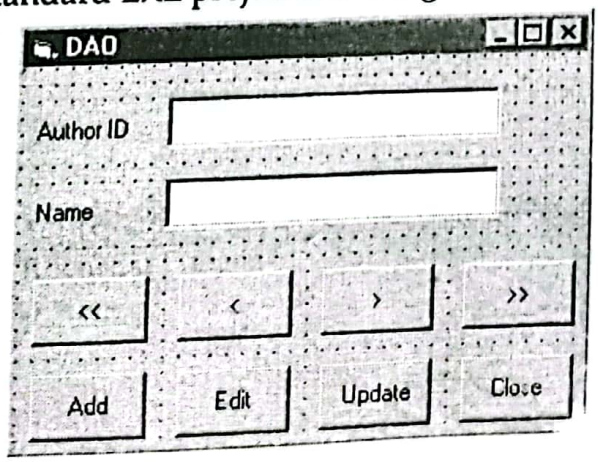
Creating a Recordset Object

You can create a **Recordset** object with DAO by using the **CreateQueryDef** method of a **Database** object, as shown in the following example.

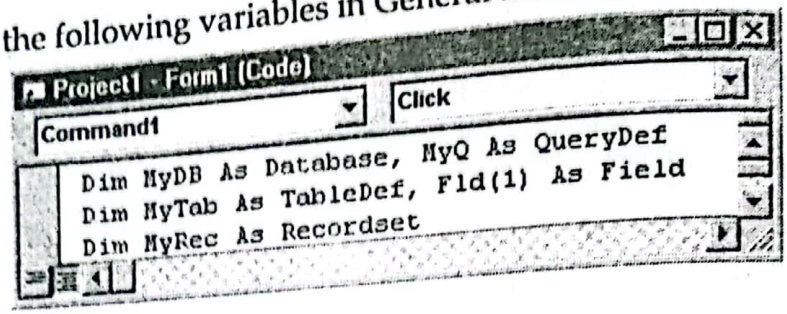
```
Dim MyDB As Database, MyQ As QueryDef
Dim strSQL As String, MyRec As Recordset
Set MyDB = OpenDatabase("IT")
Set MyRec = MyDB.OpenRecordset("Authors", dbOpenTable)
```

Practical 14.5

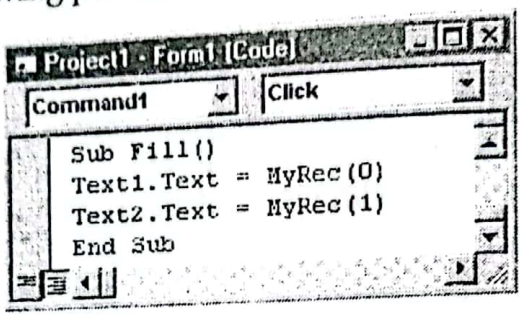
1. Start a new Standard EXE project and design the form as follows:



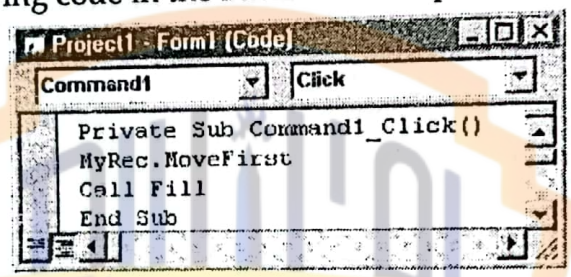
2. Declare the following variables in General Declaration section.



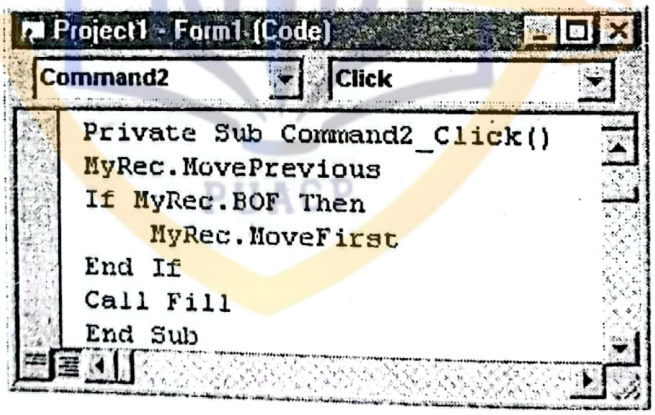
3. Declare the following procedure:



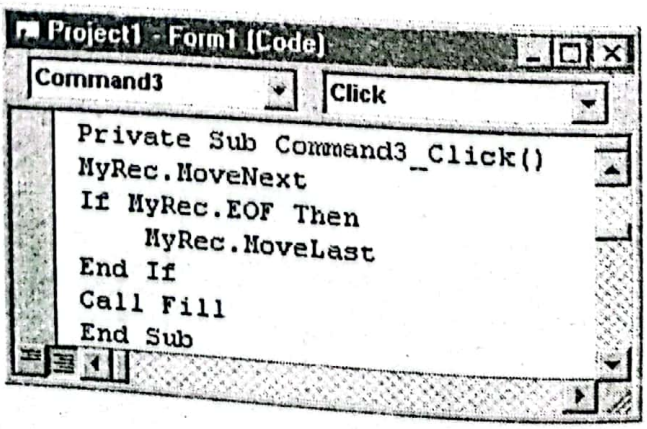
4. Add the following code in the button with caption "<<".



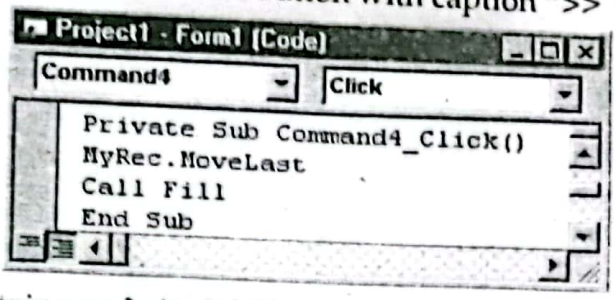
5. Add the following code in the button with caption "<".



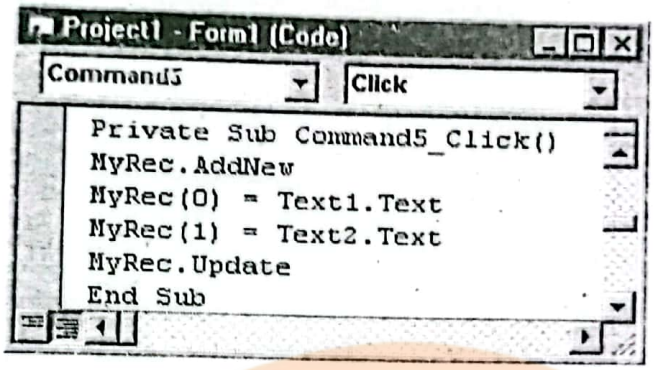
6. Add the following code in the button with caption ">".



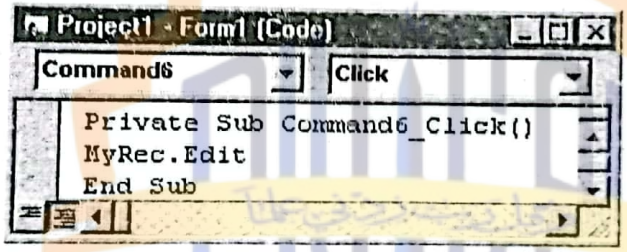
7. Add the following code in the button with caption ">>".



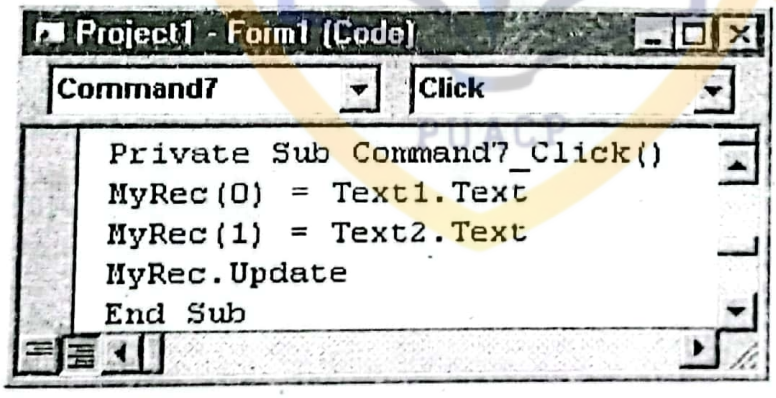
8. Add the following code in Add button.



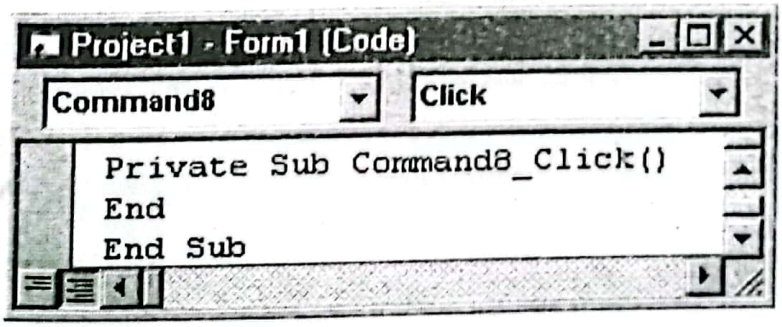
9. Add the following code in Edit button.



10. Add the following button in Update button.



11. Add the following button in Close button.



12. Run the project.

Short Questions

Q.1. What are some advantages of databases?

A database is a collection of data in an organized way. Databases provide us many advantages and benefits. We use databases to store data for the following reasons:

Efficiency: The data in databases is stored in such a way that you can retrieve the desired data efficiently. It takes less time to search your data from the database.

Less Space: The data in database is stored in such a way that it takes as much space as needed. The early file system took more space and sometimes, the data was duplicated which wasted the memory.

Data Integrity: Data integrity means reliability and accuracy of data. Enforcing data integrity ensures the quality of data in the database.

Data Security: Security is applied on the data to stop unauthorized access. Databases provide more security than file system.

Q.2. What is primary key?

A primary key is the column or a group of columns that can uniquely identify a row in a table. Some important points of a primary key are as follows:

- It must uniquely identify each record in a table.
- It must contain unique values.
- It cannot have a null value.
- Each table can have only one primary key.

Q.3. What is foreign key?

A foreign key is a copy of a primary key in another table. The key connects to another table when a relationship is being established between two tables. A table may contain many foreign keys. Some important points of a primary key are as follows:

- It has the same name as the primary key from which it was referred.
- The field specifications of the primary key from which it was copied have to be the same for the foreign key too.

Q.4. Briefly distinguish one-to-one and one-to-many relationships.

In a one-to-one relationship, for each entry identified by a primary key, there is exactly one entry in another table with the same primary key. In a one-to-many relationship, for each entry identified by a primary key there may be zero, one, or many entries in another table with the same primary key.

Q.5. What is database engine?

Database engine receives the requests of the user from user-interface and translates them into physical operations on data store. It handles all operations on the database. It lies between application and the physical database.

Q.6. Which controls can be used as data bound?

The controls that can be used as data-bound are labels, text boxes, check boxes, list boxes, combo boxes, images, picture boxes, data-bound list boxes, data-bound combo boxes and data-bound grids.

Q.7. How are Dynaset and Snapshot-type recordsets similar or different?

Both recordsets may include data from multiple tables. Dynasets are updatable but snapshots cannot be updated.

Multiple Choice

1. A collection of stored data that are managed by a database management system is called:
   a. Data set b. Database c. File d. Streamwriter
2. Each row in a table is called:
   a. Field b. Table c. Database d. Record
3. Each column in a table is called:
   a. Field b. Table c. Database d. Record
4. A field in a database table is represented by a(n):
   a. Primary key b. Record c. Row d. Column
5. The recordset type that is NOT updatable is:
   a. Table b. Dynaset c. Snapshot d. All are updatable
6. In a Data control, the property that identifies the name of the database is:
   a. RecordSource b. DataSource c. DatabaseName d. DataField
7. A Data control provides a link between database and other controls on form which are known as:
   a. Form controls b. Data-bound controls c. Linked controls d. Table-bound controls
8. Methods that position a recordset's pointer include:
   a. Move n b. MoveLast c. MoveFirst d. All
9. Which control works in- between a data-bound control on a form and a database is the:
   a. DbList b. text box c. DbGrid d. Data control
10. To create a recordset that does not allow the parent database to be updated, which recordset type should be used when opening the recordset?
    a. Table b. Dynaset c. Snapshot d. Query
11. Which of the following property of data control is used to specify the type of database?
    a. Connect b. Recordsoruce c. Shared d. Database Name
12. To move the recordset's record pointer to next record, which method should be called?
    a. FindFirst b. FindNext c. MoveFirst d. MoveNext
13. The acronym DAO stands for:
    a. Database access only b. Data access objects
    c. Data aware objects d. Data action order
14. The database object property that contains a collection with descriptions of all tables in the database is called:
    a. TableDefs b. DataTables
    c. DataDefine d. TableData

Answers

1. d	2. a	3. a	4. a
5. d	6. a	7. b	8. c
9. a	10. c	11. d	12. d
13. b	14. a		

Fill in the Blanks

1. A _____is a collection of programs that are used to create and maintain databases.
2. DBMS stands for_____.
3. The _____ handles all the operations on the database.
4. A field in a table that is a primary key in another table is referred to as a _____ key.
5. The _____ control automatically opens a database table.
6. A data-bound control's _____ property identifies the data control that is linked to a database table.
7. The _____ property of the data control determines what action the data control takes when the record pointer is moved beyond the last record in the table.
8. The recordset method that positions the record pointer on the first record is called _____.
9. _____ property of a data control identifies the table from which data will be bound to other controls on a form.
10. _____ property identifies the name of a field that will be displayed in data-bound control.
11. The database engine, known as _____, contains the necessary properties and methods that allow a Visual Basic program to manipulate database objects.
12. A(n) _____ is a series of changes made to databases that belong to a workspace.
13. To determine the name, data type, and size of each field in TableDef object, use the TableDef's _____ property.
14. The collection of open databases that provides for transaction processing and database security is called a(n) _____.
15. To close a database table, use the _____ method.
16. A RecordsetType of _____ contains data from one or more tables and does not allow changes to be written back to the database.
17. A _____ relationship exists when one record in Table A corresponds or matches one or more records in Table B.

Answers

1. Database Management System	2. Database Management System	3. Database Engine
4. Foreign Key	5. Data	6. Data Source
7. EOF Action	8. MoveFirst	9. RecordSource
10. DataField	11. DBEngine	12. Transaction
13. Field	14. Workspace	15. close
16. Snapshot	17. One-to-Many	

True / False

1. There are two commonly used types of fields: character and index.
2. In a database, only one field can be designed as key field.
3. A database is a collection of data prepared for a particular user in separate files.
4. Database records may be entered and modified

5. A record is made up of fields.
6. An advantage of database is that redundancy is reduced.
7. A database is an organized collection of related data.
8. A DBMS is a software
9. Fields widths may vary from character fields but are always the same for numeric fields.
10. In relational database, a file is also called a relation.
11. In a database, field names must be unique.
12. The data item in a field is the same for each row of relation.
13. A user enters data in database one column at a time.
14. There is exactly one file for one database.
15. A table is a grouping of related records.
16. A group of related fields is called a database.
17. A Visual Basic recordset object may be of type table.
18. A recordset object of type snapshot is updatable.
19. When using a Data control, TableSource property names the table.
20. Data-bound controls include labels and option buttons.
21. A data control's EOF property is True when record pointer is positioned past the last record.
22. Executing a recordset's MoveLast method positions the record pointer beyond the last record.
23. Dynaset-type and snapshot-type recordsets may both represent data from a single database table.
24. FindFirst method always moves the record pointer to the first record in a recordset.
25. CommitTrans method terminates the current transaction and saves the changes.
26. Microsoft's libraries of database objects are officially called data-aware objects.
27. OpenDatabase method is used to open a database.
28. DBEngine contains the properties and methods to manipulate database objects.
29. StartOver workspace method is used to end the current transactions and restore the databases to their original state.
30. After a database object is created, it is added to Databases collection in appropriately identified workspace.
31. Use FirstRecord method to move record pointer to the first record.
32. MoveNext method is used to move record pointer to the next record in the recordset.
33. One advantage of splitting data across multiple tables is that there is no redundant data.
34. Visual Basic is only equipped to handle Access databases.
35. DBType property of the Data control identifies DBMS format of the database.
36. By default, if the user types a new value in a data-bound text box, the current database record will be updated automatically.

Answers

1. F	2. F	3. F	4. F
5. T	6. T	7. T	8. T
9. F	10. T	11. T	12. F
13. F	14. F	15. T	16. F
17. T	18. F	19. F	20. F
21. T	22. F	23. T	24. F
25. T	26. F	27. T	28. T
29. F	30. T	31. F	32. F
33. F	34. F	35. F	36. T

Project 9

Phone Directory

Project Specification

The project **Phone Directory** displays a form that uses textboxes to input the following:

- First Name
- Second Name
- Phone

It stores the information in a database using Data control. It displays different buttons to navigate, add, delete, edit and update the database. It also provide the fasility to search the database by First Name, Second Name or Phone.

Procedure

1. Start MS Access and creates a new database **Phone.**
2. Create a new table **Phone** with the following fields:

Phone : Table

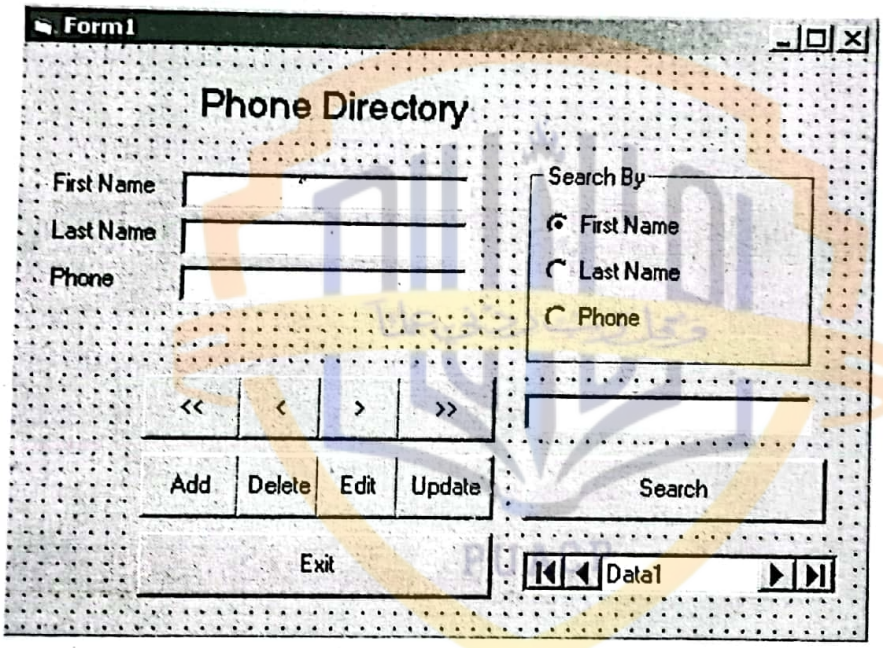
FirstName	LastName	Phone
Usman	Khalil	1234567
Amjad	Hussain	5213622
Waqar	Ahmed	7212345
Aleem	Ahmed	7654321
Nadeem	Khalil	8712327
Adnan	Ali	8733474

3. Set **Phone** field as primary key.
4. Select Tools > Database Utilities > Convert Database > To Access 97 File Format... A dialog box will appear to get new file name.
5. Enter **PhoneDir** as database name and click **Save** button. The database will be converted to Access 97 format and saved with **PhoneDir** name.
6. Start a new Standard EXE project in Visual Basic.
7. Place the following controls on the form with the given properties:

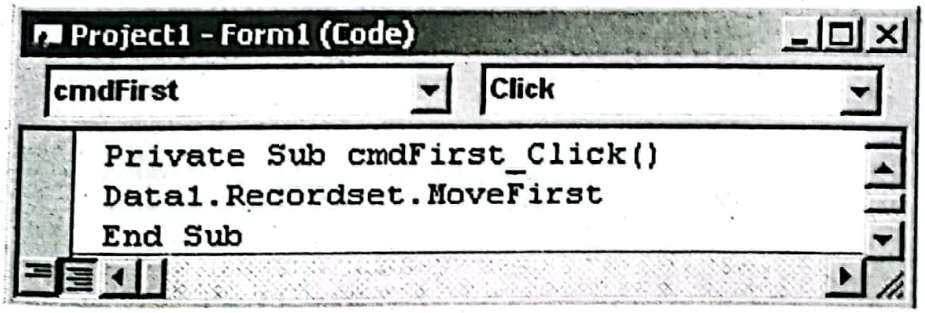
Control	Control Name	Property	Value
Label	lblTitle	Caption	Phone Directory
Label	lblFName	Caption	First Name
Label	lblLName	Caption	Last Name
Label	lblPhone	Caption	Phone
Textbox	txtFName	Text	Blank
Textbox	txtLName	Text	Blank
Textbox	txtPhone	Text	Blank
CommandButton	cmdFirst	Caption	<<
CommandButton	cmdPrevious	Caption	<
CommandButton	cmdNext	Caption	>
CommandButton	cmdLast	Caption	>>
CommandButton	cmdAdd	Caption	Add

CommandButton	cmdDelete	Caption	Delete
CommandButton	cmdEdit	Caption	Edit
CommandButton	cmdUpdate	Caption	Update
CommandButton	cmdExit	Caption	Exit
Frame	frmSearch	Caption	Search By
OptionButton	optFName	Caption	First Name
		Value	True
OptionButton	optLName	Caption	Second Name
		Value	False
OptionButton	optPhone	Caption	Phone
		Value	False
Textbox	txtSearch	Text	Blank
CommandButton	cmdSearch	Caption	Search
Data control	Data1	Visible	False

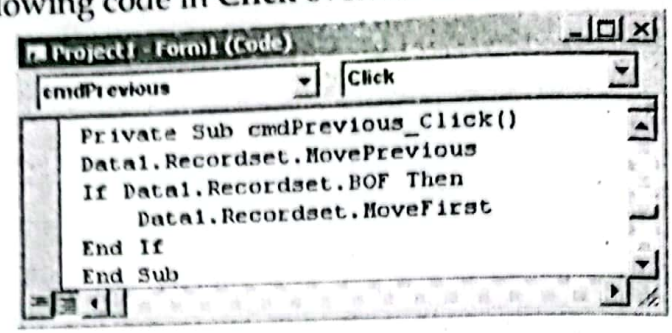
8. Design the form as follows:



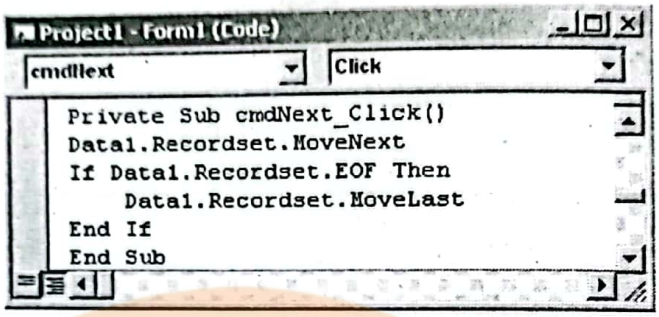
9. Set **DatabaseName** property of **Data1** to the database **PhoneDir** created above.
10. Set **RecordSource** property of **Data1** to **Phone** table.
11. Set DataSource property of txtFName, txtLName and txtPhone to Data1.
12. Set DataField property of txtFName to FirstName.
13. Set DataField property of txtLName to LastName.
14. Set DataField property of txtPhone to Phone.
15. Add the following code in **Click** even of **cmdFirst** button:



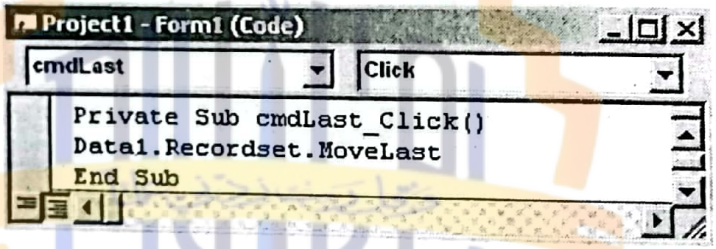
16. Add the following code in Click event of **cmdPrevious** button:



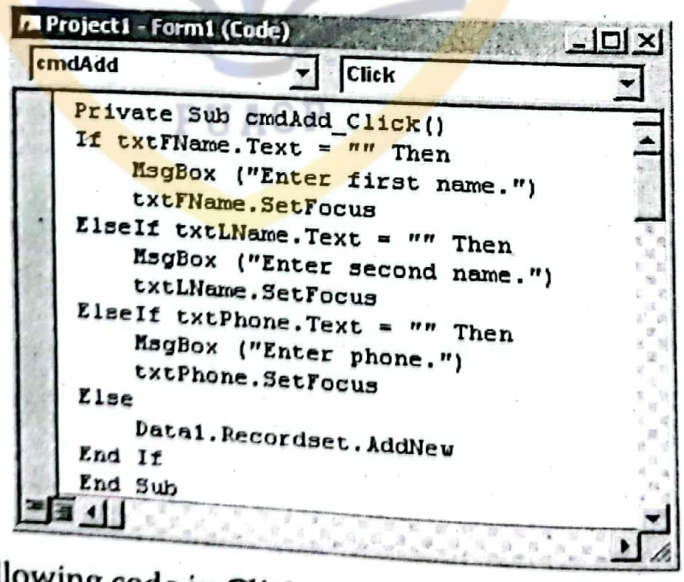
17. Add the following code in Click event of **cmdNext** button:



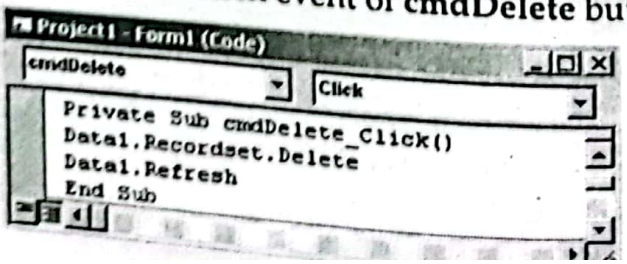
18. Add the following code in Click event of **cmdLast** button:



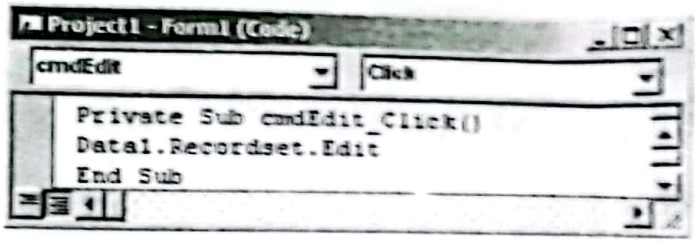
19. Add the following code in Click event of **cmdAdd** button:



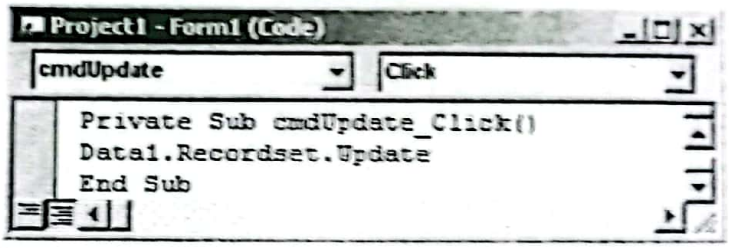
20. Add the following code in Click event of **cmdDelete** button:



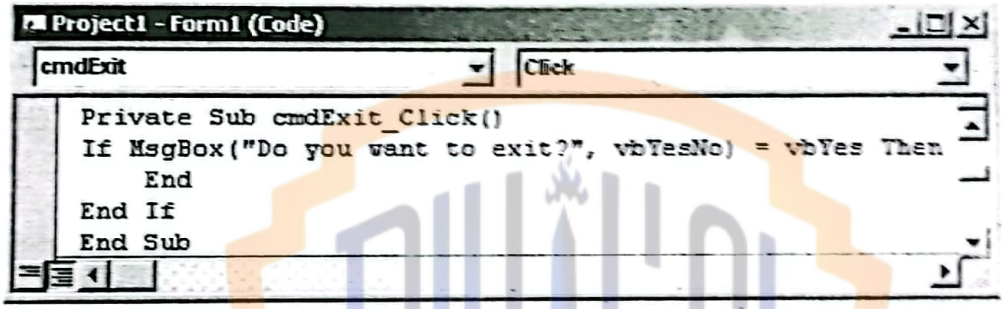
21. Add the following code in **Click** event of **cmdEdit** button:



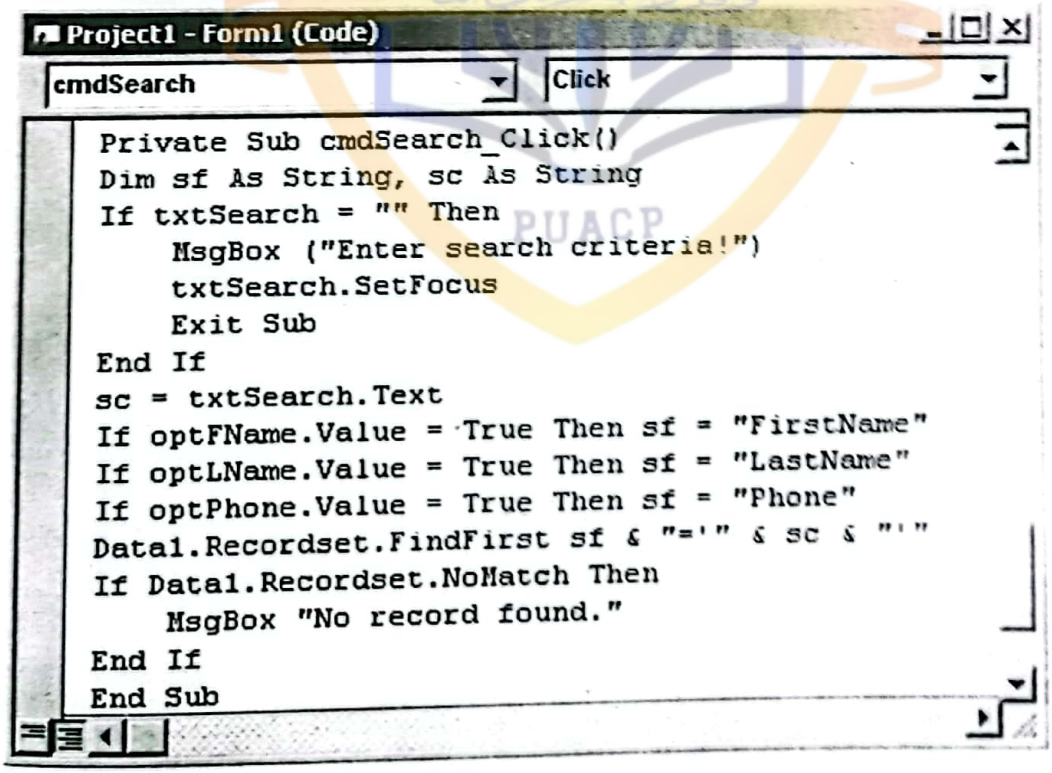
22. Add the following code in **Click** event of **cmdUpdate** button:



23. Add the following code in **Click** event of **cmdExit** button:



24. Add the following code in **Click** event of **cmdSearch** button:



25. Run the project and check its working.